

v2.0 dokumentacija

Generated by Doxygen 1.17.0

1 Hierarchical Index	1
1.1 Class Hierarchy	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Class Documentation	7
4.1 Studentas Class Reference	7
4.1.1 Detailed Description	8
4.1.2 Constructor & Destructor Documentation	8
4.1.2.1 Studentas() [1/5]	8
4.1.2.2 Studentas() [2/5]	8
4.1.2.3 Studentas() [3/5]	8
4.1.2.4 ~Studentas()	9
4.1.2.5 Studentas() [4/5]	9
4.1.2.6 Studentas() [5/5]	9
4.1.3 Member Function Documentation	9
4.1.3.1 egzam()	9
4.1.3.2 galBalas() [1/2]	9
4.1.3.3 galBalas() [2/2]	9
4.1.3.4 operator=() [1/2]	9
4.1.3.5 operator=() [2/2]	10
4.1.3.6 paz()	10
4.1.3.7 print()	10
4.1.3.8 readStudent()	10
4.1.3.9 rez()	10
4.1.3.10 skaiciuotiRez()	10
4.1.3.11 test_eilute()	11
4.2 studentas Struct Reference	11
4.2.1 Detailed Description	11
4.2.2 Member Data Documentation	11
4.2.2.1 egzam	11
4.2.2.2 pavarde	11
4.2.2.3 paz	11
4.2.2.4 rez	12
4.2.2.5 vardas	12
4.3 Zmogus Class Reference	12
4.3.1 Detailed Description	13
4.3.2 Constructor & Destructor Documentation	13
4.3.2.1 Zmogus() [1/4]	13

4.3.2.2 Zmogus() [2/4]	13
4.3.2.3 ~Zmogus()	13
4.3.2.4 Zmogus() [3/4]	13
4.3.2.5 Zmogus() [4/4]	13
4.3.3 Member Function Documentation	14
4.3.3.1 galBalas()	14
4.3.3.2 operator=() [1/2]	14
4.3.3.3 operator=() [2/2]	14
4.3.3.4 pavarde()	14
4.3.3.5 print()	14
4.3.3.6 readStudent()	14
4.3.3.7 vardas()	14
4.3.4 Friends And Related Symbol Documentation	15
4.3.4.1 operator<<	15
4.3.4.2 operator>>	15
4.3.5 Member Data Documentation	15
4.3.5.1 pavarde_	15
4.3.5.2 vardas_	15
4.4 zmogus Struct Reference	15
4.4.1 Detailed Description	16
4.4.2 Member Data Documentation	16
4.4.2.1 pavarde	16
4.4.2.2 vardas	16
5 File Documentation	17
5.1 Funkcijos.cpp File Reference	17
5.1.1 Function Documentation	17
5.1.1.1 clearInput()	17
5.1.1.2 failu_generavimas()	18
5.1.1.3 getWordInput()	18
5.1.1.4 mediana()	18
5.1.1.5 print() [1/3]	18
5.1.1.6 print() [2/3]	18
5.1.1.7 print() [3/3]	18
5.1.1.8 raide_choice()	19
5.1.1.9 rusiavimas() [1/3]	19
5.1.1.10 rusiavimas() [2/3]	19
5.1.1.11 rusiavimas() [3/3]	19
5.1.1.12 vidurkis()	19
5.2 Funkcijos.cpp	20
5.3 Funkcijos.h File Reference	22
5.3.1 Function Documentation	23

5.3.1.1 clearInput()	23
5.3.1.2 compare()	23
5.3.1.3 failu_generavimas()	23
5.3.1.4 getInput()	23
5.3.1.5 getWordInput()	23
5.3.1.6 mediana()	24
5.3.1.7 outputas() [1/3]	24
5.3.1.8 outputas() [2/3]	24
5.3.1.9 outputas() [3/3]	24
5.3.1.10 print() [1/3]	24
5.3.1.11 print() [2/3]	24
5.3.1.12 print() [3/3]	25
5.3.1.13 raide_choice()	25
5.3.1.14 rusiavimas() [1/3]	25
5.3.1.15 rusiavimas() [2/3]	25
5.3.1.16 rusiavimas() [3/3]	25
5.3.1.17 vidurkis()	25
5.4 Funkcijos.h	26
5.5 main.cpp File Reference	26
5.5.1 Function Documentation	27
5.5.1.1 main()	27
5.5.1.2 outputas()	27
5.6 main.cpp	27
5.7 Person.h File Reference	31
5.8 Person.h	31
5.9 README.md File Reference	32
5.10 Studentas.cpp File Reference	32
5.10.1 Function Documentation	32
5.10.1.1 compare()	32
5.10.1.2 comparePagalPavarde()	33
5.10.1.3 comparePagalRez()	33
5.10.1.4 mediana()	33
5.10.1.5 print()	33
5.10.1.6 rusiavimas()	33
5.10.1.7 vidurkis()	33
5.11 Studentas.cpp	34
5.12 Studentas.h File Reference	36
5.12.1 Function Documentation	36
5.12.1.1 compare()	36
5.12.1.2 comparePagalPavarde()	37
5.12.1.3 comparePagalRez()	37
5.12.1.4 mediana()	37

5.12.1.5	outputas()	37
5.12.1.6	print()	37
5.12.1.7	rusiavimas()	37
5.12.1.8	vidurkis()	38
5.13	Studentas.h	38
5.14	Test.cpp File Reference	39
5.14.1	Function Documentation	39
5.14.1.1	main()	39
5.14.1.2	TEST() [1/10]	39
5.14.1.3	TEST() [2/10]	39
5.14.1.4	TEST() [3/10]	40
5.14.1.5	TEST() [4/10]	40
5.14.1.6	TEST() [5/10]	40
5.14.1.7	TEST() [6/10]	40
5.14.1.8	TEST() [7/10]	40
5.14.1.9	TEST() [8/10]	40
5.14.1.10	TEST() [9/10]	41
5.14.1.11	TEST() [10/10]	41
5.15	Test.cpp	41
5.16	Zmones.cpp File Reference	42
5.16.1	Function Documentation	42
5.16.1.1	gen()	42
5.17	Zmones.cpp	42
5.18	Zmones.h File Reference	43
5.18.1	Function Documentation	43
5.18.1.1	gen()	43
5.19	Zmones.h	43
Index		45

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

studentas	11
Zmogus	12
Studentas	7
zmogus	15

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Studentas	7
studentas	11
Zmogus	12
zmogus	15

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

Funkcijos.cpp	17
Funkcijos.h	22
main.cpp	26
Person.h	31
Studentas.cpp	32
Studentas.h	36
Test.cpp	39
Zmones.cpp	42
Zmones.h	43

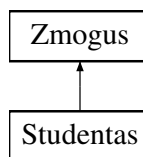
Chapter 4

Class Documentation

4.1 Studentas Class Reference

```
#include <Studentas.h>
```

Inheritance diagram for Studentas:



Public Member Functions

- **Studentas** ()
- **Studentas** (string vardas, string pavarde, vector< int > paz, double egzam)
- **Studentas** (istream &is)
- double **egzam** () const
- double **rez** () const
- const vector< int > & **paz** () const
- double **galBalas** () const override
- void **print** (ostream &os) const override
- double **galBalas** (double(*) (vector< int > &)= **mediana**) const
- void **skaiciuotiRez** (double(*) (vector< int > &)= **mediana**)
- ~**Studentas** () override
- **Studentas** (const **Studentas** &other)
- **Studentas** (**Studentas** &&other) noexcept
- **Studentas** & **operator=** (const **Studentas** &other)
- **Studentas** & **operator=** (**Studentas** &&other) noexcept
- istream & **readStudent** (istream &is) override
- string **test_eilute** ()

Public Member Functions inherited from Zmogus

- **Zmogus** ()
- **Zmogus** (string vardas, string pavarde)
- virtual string **vardas** () const
- virtual string **pavarde** () const
- virtual **~Zmogus** ()
- **Zmogus** (const **Zmogus** &other)
- **Zmogus** (**Zmogus** &&other) noexcept
- **Zmogus** & **operator=** (const **Zmogus** &other)
- **Zmogus** & **operator=** (**Zmogus** &&other) noexcept

Additional Inherited Members

Protected Attributes inherited from Zmogus

- string **vardas_**
- string **pavarde_**

4.1.1 Detailed Description

Definition at line 31 of file **Studentas.h**.

4.1.2 Constructor & Destructor Documentation

4.1.2.1 Studentas() [1/5]

```
Studentas::Studentas ()
```

Definition at line 4 of file **Studentas.cpp**.

4.1.2.2 Studentas() [2/5]

```
Studentas::Studentas (  
    string vardas,  
    string pavarde,  
    vector< int > paz,  
    double egzam)
```

Definition at line 10 of file **Studentas.cpp**.

4.1.2.3 Studentas() [3/5]

```
Studentas::Studentas (  
    istream & is)
```

Definition at line 16 of file **Studentas.cpp**.

4.1.2.4 ~Studentas()

```
Studentas::~~Studentas () [override]
```

Definition at line 99 of file **Studentas.cpp**.

4.1.2.5 Studentas() [4/5]

```
Studentas::Studentas (  
    const Studentas & other)
```

Definition at line 101 of file **Studentas.cpp**.

4.1.2.6 Studentas() [5/5]

```
Studentas::Studentas (  
    Studentas && other) [noexcept]
```

Definition at line 107 of file **Studentas.cpp**.

4.1.3 Member Function Documentation

4.1.3.1 egzam()

```
double Studentas::egzam () const [inline]
```

Definition at line 42 of file **Studentas.h**.

4.1.3.2 galBalas() [1/2]

```
double Studentas::galBalas () const [override], [virtual]
```

Implements **Zmogus** (p. 14).

Definition at line 46 of file **Studentas.cpp**.

4.1.3.3 galBalas() [2/2]

```
double Studentas::galBalas (  
    double(* strategija) (vector< int > &) = mediana) const
```

Definition at line 60 of file **Studentas.cpp**.

4.1.3.4 operator=() [1/2]

```
Studentas & Studentas::operator= (  
    const Studentas & other)
```

Definition at line 114 of file **Studentas.cpp**.

4.1.3.5 operator=() [2/2]

```
Studentas & Studentas::operator= (  
    Studentas && other) [noexcept]
```

Definition at line 123 of file **Studentas.cpp**.

4.1.3.6 paz()

```
const vector< int > & Studentas::paz () const [inline]
```

Definition at line 44 of file **Studentas.h**.

4.1.3.7 print()

```
void Studentas::print (  
    ostream & os) const [override], [virtual]
```

Implements **Zmogus** (p. 14).

Definition at line 134 of file **Studentas.cpp**.

4.1.3.8 readStudent()

```
istream & Studentas::readStudent (  
    istream & is) [override], [virtual]
```

Implements **Zmogus** (p. 14).

Definition at line 78 of file **Studentas.cpp**.

4.1.3.9 rez()

```
double Studentas::rez () const [inline]
```

Definition at line 43 of file **Studentas.h**.

4.1.3.10 skaiciuotiRez()

```
void Studentas::skaiciuotiRez (  
    double(* strategija) (vector< int > &) = mediana)
```

Definition at line 74 of file **Studentas.cpp**.

4.1.3.11 test_eilute()

```
string Studentas::test_eilute ()
```

Definition at line 172 of file **Studentas.cpp**.

The documentation for this class was generated from the following files:

- **Studentas.h**
- **Studentas.cpp**

4.2 studentas Struct Reference

```
#include <Funkcijos.h>
```

Public Attributes

- string **vardas**
- string **pavarde**
- vector< int > **paz**
- int **egzam**
- double **rez**

4.2.1 Detailed Description

Definition at line 20 of file **Funkcijos.h**.

4.2.2 Member Data Documentation

4.2.2.1 egzam

```
int studentas::egzam
```

Definition at line 24 of file **Funkcijos.h**.

4.2.2.2 pavarde

```
string studentas::pavarde
```

Definition at line 22 of file **Funkcijos.h**.

4.2.2.3 paz

```
vector<int> studentas::paz
```

Definition at line 23 of file **Funkcijos.h**.

4.2.2.4 rez

```
double studentas::rez
```

Definition at line 25 of file **Funkcijos.h**.

4.2.2.5 vardas

```
string studentas::vardas
```

Definition at line 21 of file **Funkcijos.h**.

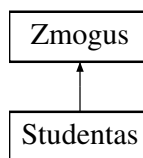
The documentation for this struct was generated from the following file:

- **Funkcijos.h**

4.3 Zmogus Class Reference

```
#include <Person.h>
```

Inheritance diagram for Zmogus:



Public Member Functions

- **Zmogus** ()
- **Zmogus** (string vardas, string pavarde)
- virtual string **vardas** () const
- virtual string **pavarde** () const
- virtual double **galBalas** () const =0
- virtual istream & **readStudent** (istream &is)=0
- virtual void **print** (ostream &os) const =0
- virtual ~**Zmogus** ()
- **Zmogus** (const **Zmogus** &other)
- **Zmogus** (**Zmogus** &&other) noexcept
- **Zmogus** & **operator=** (const **Zmogus** &other)
- **Zmogus** & **operator=** (**Zmogus** &&other) noexcept

Protected Attributes

- string **vardas_**
- string **pavarde_**

Friends

- `istream & operator>> (istream &is, Zmogus &z)`
- `ostream & operator<< (ostream &os, const Zmogus &z)`

4.3.1 Detailed Description

Definition at line 11 of file **Person.h**.

4.3.2 Constructor & Destructor Documentation

4.3.2.1 Zmogus() [1/4]

```
Zmogus::Zmogus () [inline]
```

Definition at line 17 of file **Person.h**.

4.3.2.2 Zmogus() [2/4]

```
Zmogus::Zmogus (  
    string vardas,  
    string pavarde) [inline]
```

Definition at line 18 of file **Person.h**.

4.3.2.3 ~Zmogus()

```
virtual Zmogus::~Zmogus () [inline], [virtual]
```

Definition at line 27 of file **Person.h**.

4.3.2.4 Zmogus() [3/4]

```
Zmogus::Zmogus (  
    const Zmogus & other) [inline]
```

Definition at line 29 of file **Person.h**.

4.3.2.5 Zmogus() [4/4]

```
Zmogus::Zmogus (  
    Zmogus && other) [inline], [noexcept]
```

Definition at line 32 of file **Person.h**.

4.3.3 Member Function Documentation

4.3.3.1 galBalas()

```
virtual double Zmogus::galBalas () const [pure virtual]
```

Implemented in **Studentas** (p. 9).

4.3.3.2 operator=() [1/2]

```
Zmogus & Zmogus::operator= (
    const Zmogus & other) [inline]
```

Definition at line 35 of file **Person.h**.

4.3.3.3 operator=() [2/2]

```
Zmogus & Zmogus::operator= (
    Zmogus && other) [inline], [noexcept]
```

Definition at line 41 of file **Person.h**.

4.3.3.4 pavarde()

```
virtual string Zmogus::pavarde () const [inline], [virtual]
```

Definition at line 21 of file **Person.h**.

4.3.3.5 print()

```
virtual void Zmogus::print (
    ostream & os) const [pure virtual]
```

Implemented in **Studentas** (p. 10).

4.3.3.6 readStudent()

```
virtual istream & Zmogus::readStudent (
    istream & is) [pure virtual]
```

Implemented in **Studentas** (p. 10).

4.3.3.7 vardas()

```
virtual string Zmogus::vardas () const [inline], [virtual]
```

Definition at line 20 of file **Person.h**.

4.3.4 Friends And Related Symbol Documentation

4.3.4.1 operator<<

```
ostream & operator<< (  
    ostream & os,  
    const Zmogus & z) [friend]
```

Definition at line 51 of file **Person.h**.

4.3.4.2 operator>>

```
istream & operator>> (  
    istream & is,  
    Zmogus & z) [friend]
```

Definition at line 48 of file **Person.h**.

4.3.5 Member Data Documentation

4.3.5.1 pavarde_

```
string Zmogus::pavarde_ [protected]
```

Definition at line 14 of file **Person.h**.

4.3.5.2 vardas_

```
string Zmogus::vardas_ [protected]
```

Definition at line 13 of file **Person.h**.

The documentation for this class was generated from the following file:

- **Person.h**

4.4 zmogus Struct Reference

```
#include <Zmones.h>
```

Public Attributes

- string **vardas**
- string **pavarde**

4.4.1 Detailed Description

Definition at line 10 of file **Zmones.h**.

4.4.2 Member Data Documentation

4.4.2.1 pavarde

```
string zmogus::pavarde
```

Definition at line 12 of file **Zmones.h**.

4.4.2.2 vardas

```
string zmogus::vardas
```

Definition at line 11 of file **Zmones.h**.

The documentation for this struct was generated from the following file:

- **Zmones.h**

Chapter 5

File Documentation

5.1 Funkcijos.cpp File Reference

```
#include <iostream>
#include <algorithm>
#include <numeric>
#include <fstream>
#include <iomanip>
#include <chrono>
#include <stdexcept>
#include "Funkcijos.h"
```

Functions

- void **clearInput** ()
- void **vidurkis** (**studentas** &S)
- void **mediana** (**studentas** &S)
- void **rusiavimas** (vector< **studentas** > &A, int sort_choice)
- void **rusiavimas** (deque< **studentas** > &A, int sort_choice)
- void **rusiavimas** (list< **studentas** > &A, int sort_choice)
- string **raide_choice** (const string &prompt1)
- string **getWordInput** (const string &prompt1, const string &prompt2)
- void **print** (std::ostream &os, int pasirinkimas, vector< **studentas** > &A, int stud_skaicius)
- void **print** (std::ostream &os, int pasirinkimas, deque< **studentas** > &A, int stud_skaicius)
- void **print** (std::ostream &os, int pasirinkimas, list< **studentas** > &A, int stud_skaicius)
- void **failu_generavimas** (const string &filename)

5.1.1 Function Documentation

5.1.1.1 clearInput()

```
void clearInput ()
```

Definition at line **26** of file **Funkcijos.cpp**.

5.1.1.2 failu_generavimas()

```
void failu_generavimas (  
    const string & filename)
```

Definition at line 187 of file **Funkcijos.cpp**.

5.1.1.3 getWordInput()

```
string getWordInput (  
    const string & prompt1,  
    const string & prompt2)
```

Definition at line 134 of file **Funkcijos.cpp**.

5.1.1.4 mediana()

```
void mediana (  
    studentas & S)
```

Definition at line 45 of file **Funkcijos.cpp**.

5.1.1.5 print() [1/3]

```
void print (  
    std::ostream & os,  
    int pasirinkimas,  
    deque< studentas > & A,  
    int stud_skaicius)
```

Definition at line 164 of file **Funkcijos.cpp**.

5.1.1.6 print() [2/3]

```
void print (  
    std::ostream & os,  
    int pasirinkimas,  
    list< studentas > & A,  
    int stud_skaicius)
```

Definition at line 173 of file **Funkcijos.cpp**.

5.1.1.7 print() [3/3]

```
void print (  
    std::ostream & os,  
    int pasirinkimas,  
    vector< studentas > & A,  
    int stud_skaicius)
```

Definition at line 155 of file **Funkcijos.cpp**.

5.1.1.8 raide_choice()

```
string raide_choice (  
    const string & prompt1)
```

Definition at line 116 of file **Funkcijos.cpp**.

5.1.1.9 rusiavimas() [1/3]

```
void rusiavimas (  
    deque< studentas > & A,  
    int sort_choice)
```

Definition at line 80 of file **Funkcijos.cpp**.

5.1.1.10 rusiavimas() [2/3]

```
void rusiavimas (  
    list< studentas > & A,  
    int sort_choice)
```

Definition at line 98 of file **Funkcijos.cpp**.

5.1.1.11 rusiavimas() [3/3]

```
void rusiavimas (  
    vector< studentas > & A,  
    int sort_choice)
```

Definition at line 62 of file **Funkcijos.cpp**.

5.1.1.12 vidurkis()

```
void vidurkis (  
    studentas & S)
```

Definition at line 32 of file **Funkcijos.cpp**.

5.2 Funkcijos.cpp

Go to the documentation of this file.

```

00001 #include <iostream>
00002 #include <algorithm>
00003 #include <numeric>
00004 #include <fstream>
00005 #include <iomanip>
00006 #include <chrono>
00007 #include <stdexcept>
00008 #include "Funkcijos.h"
00009
00010 using std::cout;
00011 using std::cin;
00012 using std::sort;
00013 using std::vector;
00014 using std::deque;
00015 using std::list;
00016 using std::string;
00017 using std::endl;
00018 using std::left;
00019 using std::setw;
00020 using std::fixed;
00021 using std::setprecision;
00022 using std::chrono::high_resolution_clock;
00023 using std::chrono::duration;
00024 using std::ofstream;
00025
00026 void clearInput() {
00027     cout << "Neteisinga ivestis. ";
00028     cin.clear();
00029     cin.ignore(10000, '\n');
00030 }
00031
00032 void vidurkis(studentas &S) {
00033     if (S.paz.empty()) {
00034         S.rez = S.egzam * 0.6;
00035     }
00036     else {
00037         double sum = 0;
00038         for(int j = 0; j < S.paz.size(); j++) {
00039             sum += S.paz[j];
00040         }
00041         S.rez = sum / S.paz.size() * 0.4 + S.egzam * 0.6;
00042     }
00043 }
00044
00045 void mediana(studentas &S) {
00046     double mediana;
00047     if (S.paz.empty()) {
00048         S.rez = S.egzam * 0.6;
00049     }
00050     else {
00051         sort(S.paz.begin(), S.paz.end());
00052         if(S.paz.size() % 2 == 0) {
00053             mediana = (S.paz[S.paz.size() / 2 - 1] + S.paz[S.paz.size() / 2]) / 2.0;
00054         }
00055         else {
00056             mediana = S.paz[S.paz.size() / 2];
00057         }
00058         S.rez = mediana * 0.4 + S.egzam * 0.6;
00059     }
00060 }
00061
00062 void rusiavimas(vector<studentas> &A, int sort_choice) {
00063     if(sort_choice == 1) {
00064         sort(A.begin(), A.end(), [](studentas& a, studentas& b) {
00065             return compare(a.vardas, b.vardas);
00066         });
00067     }
00068     else if(sort_choice == 2) {
00069         sort(A.begin(), A.end(), [](studentas& a, studentas& b) {
00070             return compare(a.pavarde, b.pavarde);
00071         });
00072     }
00073     else {
00074         sort(A.begin(), A.end(), [](studentas& a, studentas& b) {
00075             return compare(b.rez, a.rez);
00076         });
00077     }
00078 }
00079
00080 void rusiavimas(deque<studentas> &A, int sort_choice) {
00081     if(sort_choice == 1) {
00082         sort(A.begin(), A.end(), [](studentas& a, studentas& b) {

```

```

00083         return compare(a.vardas, b.vardas);
00084     });
00085 }
00086 else if(sort_choice == 2) {
00087     sort(A.begin(), A.end(), [](studentas& a, studentas& b) {
00088         return compare(a.pavarde, b.pavarde);
00089     });
00090 }
00091 else {
00092     sort(A.begin(), A.end(), [](studentas& a, studentas& b) {
00093         return compare(b.rez, a.rez);
00094     });
00095 }
00096 }
00097
00098 void rusiavimas(list<studentas> &A, int sort_choice) {
00099     if(sort_choice == 1) {
00100         A.sort([](studentas& a, studentas& b) {
00101             return compare(a.vardas, b.vardas);
00102         });
00103     }
00104     else if(sort_choice == 2) {
00105         A.sort([](studentas& a, studentas& b) {
00106             return compare(a.pavarde, b.pavarde);
00107         });
00108     }
00109     else {
00110         A.sort([](studentas& a, studentas& b) {
00111             return compare(b.rez, a.rez);
00112         });
00113     }
00114 }
00115
00116 string raide_choice(const string& prompt1) {
00117     string choice;
00118     while(true) {
00119         try {
00120             cin >> choice;
00121             if(choice != "t" && choice != "T" && choice != "n" && choice != "N") {
00122                 throw std::runtime_error(prompt1);
00123             }
00124             return choice;
00125         }
00126         catch (std::exception& e) {
00127             clearInput();
00128             cout << e.what() << ": ";
00129             continue;
00130         }
00131     }
00132 }
00133
00134 string getWordInput(const string& prompt1, const string& prompt2) {
00135     string input;
00136     while(true) {
00137         try {
00138             cout << prompt1;
00139             cin >> input;
00140             for(char c : input) {
00141                 if(!isalpha(c)) {
00142                     throw std::runtime_error(prompt2);
00143                 }
00144             }
00145             return input;
00146         }
00147         catch (std::exception& e) {
00148             clearInput();
00149             cout << e.what() << endl;
00150             continue;
00151         }
00152     }
00153 }
00154
00155 void print(std::ostream& os, int pasirinkimas, vector<studentas>& A, int stud_skaicius) {
00156     os << left << setw(15) << "Vardas" << left << setw(20) << "Pavarde";
00157     (pasirinkimas == 1)? os << setw(10) << "Galutinis (Vid.)" << endl : os << setw(10) << "Galutinis
(Med.)" << endl;
00158     for(int i = 0; i < stud_skaicius; i++) {
00159         os << left << setw(15) << A[i].vardas << left << setw(20) << A[i].pavarde;
00160         os << setw(10) << fixed << setprecision(2) << A[i].rez << endl;
00161     }
00162 }
00163
00164 void print(std::ostream& os, int pasirinkimas, deque<studentas>& A, int stud_skaicius) {
00165     os << left << setw(15) << "Vardas" << left << setw(20) << "Pavarde";
00166     (pasirinkimas == 1)? os << setw(10) << "Galutinis (Vid.)" << endl : os << setw(10) << "Galutinis
(Med.)" << endl;
00167     for(int i = 0; i < stud_skaicius; i++) {

```

```

00168         os << left << setw(15) << A[i].vardas << left << setw(20) << A[i].pavarde;
00169         os << setw(10) << fixed << setprecision(2) << A[i].rez << endl;
00170     }
00171 }
00172
00173 void print(std::ostream& os, int pasirinkimas, list<studentas>& A, int stud_skaicius) {
00174     os << left << setw(15) << "Vardas" << left << setw(20) << "Pavarde";
00175     (pasirinkimas == 1)? os << setw(10) << "Galutinis (Vid.)" << endl : os << setw(10) << "Galutinis
(Med.)" << endl;
00176     int i = 0;
00177     for(studentas& s : A) {
00178         if(i >= stud_skaicius) {
00179             break;
00180         }
00181         os << left << setw(15) << s.vardas << left << setw(20) << s.pavarde;
00182         os << setw(10) << fixed << setprecision(2) << s.rez << endl;
00183         i++;
00184     }
00185 }
00186
00187 void failu_generavimas(const string& filename) {
00188     int dydis = stoi(filename);
00189     auto start = high_resolution_clock::now();
00190     ofstream fout(filename);
00191     fout << setw(16) << left << "Vardas" << setw(17) << left << "Pavarde" << setw(10) << "ND1" << setw(10) <<
"ND2" << setw(10) << "ND3" << setw(10) << "ND4" << setw(10) << "ND5" << setw(10) << "Egzaminas" << endl;
00192     for(int i = 0; i < dydis; i++) {
00193         fout << "Vardas" << setw(10) << left << i + 1 << "Pavarde" << setw(10) << left << i + 1;
00194         for(int j = 0; j < 5; j++) {
00195             fout << setw(10) << left << rand() % 10 + 1;
00196         }
00197         fout << setw(10) << left << rand() % 10 + 1 << endl;
00198     }
00199     fout.close();
00200     auto end = high_resolution_clock::now();
00201     duration<double> diff = end - start;
00202     cout << "Failo generavimo laikas: " << diff.count() << endl;
00203 }

```

5.3 Funkcijos.h File Reference

```

#include <iostream>
#include <vector>
#include <deque>
#include <list>
#include <string>
#include <stdexcept>
#include <climits>

```

Classes

- struct **studentas**

Functions

- void **clearInput** ()
- void **vidurkis** (**studentas** &S)
- void **mediana** (**studentas** &S)
- void **rusiavimas** (vector< **studentas** > &A, int sort_choice)
- void **rusiavimas** (deque< **studentas** > &A, int sort_choice)
- void **rusiavimas** (list< **studentas** > &A, int sort_choice)
- void **outputas** (vector< **studentas** > &A, int stud_skaicius, double test_time, string filename)
- void **outputas** (deque< **studentas** > &A, int stud_skaicius, double test_time, string filename)
- void **outputas** (list< **studentas** > &A, int stud_skaicius, double test_time, string filename)

- string **raide_choice** (const string &prompt1)
- string **getWordInput** (const string &prompt1, const string &prompt2)
- void **print** (std::ostream &os, int pasirinkimas, vector< **studentas** > &A, int stud_skaicius)
- void **print** (std::ostream &os, int pasirinkimas, deque< **studentas** > &A, int stud_skaicius)
- void **print** (std::ostream &os, int pasirinkimas, list< **studentas** > &A, int stud_skaicius)
- void **failu_generavimas** (const string &filename)
- template<typename T>
bool **compare** (const T &a, const T &b)
- template<typename T, int Arg1 = INT_MIN, int Arg2 = INT_MAX>
T **getInput** (const string &prompt1, const string &prompt2)

5.3.1 Function Documentation

5.3.1.1 clearInput()

```
void clearInput ()
```

Definition at line **26** of file **Funkcijos.cpp**.

5.3.1.2 compare()

```
template<typename T>  
bool compare (  
    const T & a,  
    const T & b)
```

Definition at line **46** of file **Funkcijos.h**.

5.3.1.3 failu_generavimas()

```
void failu_generavimas (  
    const string & filename)
```

Definition at line **187** of file **Funkcijos.cpp**.

5.3.1.4 getInput()

```
template<typename T, int Arg1 = INT_MIN, int Arg2 = INT_MAX>  
T getInput (  
    const string & prompt1,  
    const string & prompt2)
```

Definition at line **51** of file **Funkcijos.h**.

5.3.1.5 getWordInput()

```
string getWordInput (  
    const string & prompt1,  
    const string & prompt2)
```

Definition at line **134** of file **Funkcijos.cpp**.

5.3.1.6 mediana()

```
void mediana (  
    studentas & S)
```

Definition at line 45 of file **Funkcijos.cpp**.

5.3.1.7 outputas() [1/3]

```
void outputas (  
    deque< studentas > & A,  
    int stud_skaicius,  
    double test_time,  
    string filename)
```

5.3.1.8 outputas() [2/3]

```
void outputas (  
    list< studentas > & A,  
    int stud_skaicius,  
    double test_time,  
    string filename)
```

5.3.1.9 outputas() [3/3]

```
void outputas (  
    vector< studentas > & A,  
    int stud_skaicius,  
    double test_time,  
    string filename)
```

5.3.1.10 print() [1/3]

```
void print (  
    std::ostream & os,  
    int pasirinkimas,  
    deque< studentas > & A,  
    int stud_skaicius)
```

Definition at line 164 of file **Funkcijos.cpp**.

5.3.1.11 print() [2/3]

```
void print (  
    std::ostream & os,  
    int pasirinkimas,  
    list< studentas > & A,  
    int stud_skaicius)
```

Definition at line 173 of file **Funkcijos.cpp**.

5.3.1.12 print() [3/3]

```
void print (
    std::ostream & os,
    int pasirinkimas,
    vector< studentas > & A,
    int stud_skaicius)
```

Definition at line 155 of file **Funkcijos.cpp**.

5.3.1.13 raide_choice()

```
string raide_choice (
    const string & prompt1)
```

Definition at line 116 of file **Funkcijos.cpp**.

5.3.1.14 rusiavimas() [1/3]

```
void rusiavimas (
    deque< studentas > & A,
    int sort_choice)
```

Definition at line 80 of file **Funkcijos.cpp**.

5.3.1.15 rusiavimas() [2/3]

```
void rusiavimas (
    list< studentas > & A,
    int sort_choice)
```

Definition at line 98 of file **Funkcijos.cpp**.

5.3.1.16 rusiavimas() [3/3]

```
void rusiavimas (
    vector< studentas > & A,
    int sort_choice)
```

Definition at line 62 of file **Funkcijos.cpp**.

5.3.1.17 vidurkis()

```
void vidurkis (
    studentas & S)
```

Definition at line 32 of file **Funkcijos.cpp**.

5.4 Funkcijos.h

Go to the documentation of this file.

```

00001 #ifndef FUNKCIJOS_H
00002 #define FUNKCIJOS_H
00003
00004 #include <iostream>
00005 #include <vector>
00006 #include <deque>
00007 #include <list>
00008 #include <string>
00009 #include <stdexcept>
00010 #include <climits>
00011
00012 using std::vector;
00013 using std::deque;
00014 using std::list;
00015 using std::string;
00016 using std::cin;
00017 using std::cout;
00018 using std::endl;
00019
00020 struct studentas {
00021     string vardas;
00022     string pavarde;
00023     vector<int> paz;
00024     int egzam;
00025     double rez;
00026 };
00027
00028 void clearInput();
00029 void vidurkis(studentas& S);
00030 void mediana(studentas& S);
00031 void rusiavimas(vector<studentas>& A, int sort_choice);
00032 void rusiavimas(deque<studentas>& A, int sort_choice);
00033 void rusiavimas(list<studentas>& A, int sort_choice);
00034 void outputas(vector<studentas>& A, int stud_skaicius, double test_time, string filename);
00035 void outputas(deque<studentas>& A, int stud_skaicius, double test_time, string filename);
00036 void outputas(list<studentas>& A, int stud_skaicius, double test_time, string filename);
00037 string raide_choice(const string& prompt1);
00038 string getWordInput(const string& prompt1, const string& prompt2);
00039 void print(std::ostream& os, int pasirinkimas, vector<studentas>& A, int stud_skaicius);
00040 void print(std::ostream& os, int pasirinkimas, deque<studentas>& A, int stud_skaicius);
00041 void print(std::ostream& os, int pasirinkimas, list<studentas>& A, int stud_skaicius);
00042 void failu_generavimas(const string& filename);
00043
00044
00045 template <typename T>
00046 bool compare(const T& a, const T& b) {
00047     return a < b;
00048 }
00049
00050 template <typename T, int Arg1 = INT_MIN, int Arg2 = INT_MAX>
00051 T getInput(const string& prompt1, const string& prompt2) {
00052     T input;
00053     while(true) {
00054         try {
00055             cout << prompt1;
00056             cin >> input;
00057             if(cin.fail() || input < Arg1 || input > Arg2) {
00058                 throw std::runtime_error(prompt2);
00059             }
00060             return input;
00061         }
00062         catch(std::exception& e) {
00063             clearInput();
00064             cout << e.what() << endl;
00065         }
00066     }
00067 }
00068
00069 #endif

```

5.5 main.cpp File Reference

```

#include <iostream>
#include <iomanip>
#include <string>

```



```
#include <vector>
#include <algorithm>
#include <cctype>
#include <ctime>
#include <fstream>
#include <sstream>
#include <chrono>
#include <iterator>
#include <cstdlib>
#include "Zmones.h"
#include "Studentas.h"
#include "Funkcijos.h"
```

Functions

- `int main ()`
- `void outputas (vector< Studentas > &A, int stud_skaicius, double test_time, string filename)`

5.5.1 Function Documentation

5.5.1.1 main()

```
int main ()
```

Definition at line **33** of file **main.cpp**.

5.5.1.2 outputas()

```
void outputas (
    vector< Studentas > & A,
    int stud_skaicius,
    double test_time,
    string filename)
```

Definition at line **184** of file **main.cpp**.

5.6 main.cpp

Go to the documentation of this file.

```
00001 #include <iostream>
00002 #include <iomanip>
00003 #include <string>
00004 #include <vector>
00005 #include <algorithm>
00006 #include <cctype>
00007 #include <ctime>
00008 #include <fstream>
00009 #include <sstream>
00010 #include <chrono>
00011 #include <iterator>
00012 #include <cstdlib>
00013 #include "Zmones.h"
00014 #include "Studentas.h"
```

```

00015 #include "Funkcijos.h"
00016
00017 using std::string;
00018 using std::vector;
00019 using std::cout;
00020 using std::cin;
00021 using std::setw;
00022 using std::left;
00023 using std::endl;
00024 using std::ifstream;
00025 using std::ofstream;
00026 using std::getline;
00027 using std::sort;
00028 using std::stringstream;
00029 using std::chrono::high_resolution_clock;
00030 using std::chrono::duration;
00031
00032
00033 int main() {
00034     srand(time(0));
00035     system("dir *.txt");
00036     std::ios::sync_with_stdio(false);
00037     vector<Studentas> A;
00038     int temp, stud_skaicius = 0, index = 0;
00039     string f_choice, filename;
00040     cout << "Ar noretumet skaityti duomenis is failo? (t/n) ";
00041     f_choice = raide_choice(
00042         "Iveskite t arba n"
00043     );
00044     if(f_choice == "t" || f_choice == "T") {
00045         while(true) {
00046             cout << "Iveskite failo pavadinima: ";
00047             cin >> filename;
00048             try {
00049                 ifstream test(filename);
00050                 if(!test) {
00051                     throw std::runtime_error("Failas nerastas.");
00052                 }
00053                 break;
00054             }
00055             catch (std::exception& e) {
00056                 cout << "Klaida: " << e.what() << " Pabandykite dar karta." << endl;
00057             }
00058         }
00059         A.reserve(10000000);
00060         string line;
00061         ifstream fin(filename);
00062         getline(fin, line);
00063         stringstream ss;
00064         auto start = high_resolution_clock::now();
00065         while(getline(fin, line)) {
00066             ss.clear();
00067             ss.str(line);
00068             Studentas S;
00069             ss >> S;
00070             A.push_back(move(S));
00071             stud_skaicius++;
00072         }
00073         fin.close();
00074         auto end = high_resolution_clock::now();
00075         duration<double> diff = end - start;
00076         outputas(A, stud_skaicius, diff.count(), filename);
00077     }
00078     else {
00079         int eiga;
00080         cout << "Iveskite eiga: " << endl;
00081         cout << "1 - ranka " << endl;
00082         cout << "2 - generuoti tik pazymius " << endl;
00083         cout << "3 - generuoti studentu vardus, pavardes ir pazymius " << endl;
00084         cout << "4 - baigti darba " << endl;
00085         cout << "5 - failo generavimas " << endl;
00086         eiga = getInput<int,1,5>("Jusu pasirinkimas: ",
00087             "Iveskite skaiciu nuo 1 iki 5."
00088         );
00089         if(eiga == 4) {
00090             cout << "Darbas baigtas.";
00091             return 0;
00092         }
00093         if(eiga == 5) {
00094             for(int i = 0; i < 5; i++) {
00095                 string gen_failas;
00096                 cout << "Iveskite kiek irasu faile noretumet: ";
00097                 cin >> gen_failas;
00098                 failu_generavimas(gen_failas + ".txt");
00099             }
00100             return 0;
00101         }

```

```

00102     }
00103     stud_skaicius = getInput<int,1>(
00104         "Kiek yra studentu? ",
00105         "Iveskite teigiama skaiciu."
00106     );
00107     while(true) {
00108         string vard, pavard;
00109         vector<int> paz_temp;
00110         int egz_temp;
00111         if(eiga == 1 || eiga == 2) {
00112             vard = getWordInput(
00113                 "Iveskite varda: ",
00114                 "Vardas turi buti sudarytas tik is raidziu."
00115             );
00116             pavard = getWordInput(
00117                 "Iveskite pavarde: ",
00118                 "Pavarde turi buti sudaryta tik is raidziu."
00119             );
00120         }
00121         else if(eiga == 3) {
00122             zmogus z = gen();
00123             vard = z.vardas;
00124             pavard = z.pavarde;
00125             cout << "Sugeneruotas zmogus: " << vard << " " << pavard << endl;
00126         }
00127         if(eiga == 1) {
00128             while(true) {
00129                 temp = getInput<int,0,10>(
00130                     "Iveskite " + std::to_string(paz_temp.size() + 1) + " semestro pazymi (0 -
00131 baigti): ",
00132                     "Iveskite skaiciu tarp 0 ir 10"
00133                 );
00134                 if(temp == 0) break;
00135                 paz_temp.push_back(temp);
00136             }
00137             egz_temp = getInput<int,1,10>(
00138                 "Iveskite egzamino pazymi: ",
00139                 "Iveskite skaiciu tarp 1 ir 10."
00140             );
00141         }
00142         else if(eiga == 2 || eiga == 3) {
00143             int paz_kiek;
00144             paz_kiek = getInput<int,0>(
00145                 "Kiek pazymiu sugeneruoti? ",
00146                 "Iveskite teigiama sveika skaiciu."
00147             );
00148             for(int i = 0; i < paz_kiek; i++) {
00149                 temp = rand() % 10 + 1;
00150                 cout << i+1 << " Sugeneruotas pazymys: " << temp << endl;
00151                 paz_temp.push_back(temp);
00152             }
00153             egz_temp = rand() % 10 + 1;
00154             cout << "Sugeneruotas egzamino pazymys: " << egz_temp << endl;
00155         }
00156         stringstream ss;
00157         ss << vard << " " << pavard;
00158         for(int i = 0; i < paz_temp.size(); i++) {
00159             ss << " " << paz_temp[i];
00160         }
00161         ss << " " << egz_temp;
00162         Studentas S;
00163         ss >> S;
00164         A.push_back(move(S));
00165         string student_choice;
00166         while(true) {
00167             if(index < stud_skaicius - 1) break;
00168             else {
00169                 cout << "Ar noretumet ivesti dar viena studenta? (t/n) ";
00170                 student_choice = raide_choice(
00171                     "Iveskite t arba n"
00172                 );
00173                 if(student_choice == "t" || student_choice == "T") {
00174                     stud_skaicius++;
00175                 }
00176                 break;
00177             }
00178             index++;
00179             if(index >= stud_skaicius) break;
00180         }
00181         outputas(A, stud_skaicius, 0, "");
00182     }
00183 }
00184 void outputas(vector<Studentas>& A, int stud_skaicius, double test_time, string filename) {
00185     int grade_choice;
00186     grade_choice = getInput<int,1,2>(
00187         "Ivesti vidurki ar mediana? (1 - vidurkis, 2 - mediana) ",

```

```

00188         "Iveskite 1 arba 2."
00189     );
00190     double (*strategija)(vector<int>&);
00191     (grade_choice == 1)? strategija = vidurkis : strategija = mediana;
00192
00193     for(int i = 0; i < stud_skaicius; i++) {
00194         A[i].skaiciuotiRez(strategija);
00195     }
00196     cout << "Ar norite atskirti vargsiukus nuo kietiaku? (t/n) ";
00197     string divide_choice;
00198     divide_choice = raide_choice(
00199         "Iveskite t arba n"
00200     );
00201     int sort_choice;
00202     sort_choice = getInput<int,1,3>(
00203         "Kaip surusiuoti rezultatus? (1 - pagal varda, 2 - pagal pavarde, 3 - pagal galutini bala) ",
00204         "Iveskite 1, 2 arba 3."
00205     );
00206     if(divide_choice == "t" || divide_choice == "T") {
00207         int divide_strategy;
00208         cout << "Kuria skaidymo strategija norite naudoti? " << endl;
00209         cout << "1 - Du nauji konteineriai " << endl;
00210         cout << "2 - Vienas naujas konteineris " << endl;
00211         cout << "3 - Efektyvus darbus su konteineriais " << endl;
00212         divide_strategy = getInput<int,1,3>(
00213             "Jusu pasirinkimas: ",
00214             "Iveskite 1, 2 arba 3."
00215         );
00216         auto start1 = high_resolution_clock::now();
00217         rusiavimas(A, sort_choice);
00218         auto end1 = high_resolution_clock::now();
00219         duration<double> diff1 = end1 - start1;
00220         vector<Studentas> vargsiukai;
00221         vector<Studentas> kietiakai;
00222         auto start2 = high_resolution_clock::now();
00223         if(divide_strategy == 1) {
00224             vargsiukai.reserve(6000000);
00225             kietiakai.reserve(6000000);
00226             for(int i = 0; i < stud_skaicius; i++) {
00227                 if(A[i].rez() < 5) {
00228                     vargsiukai.push_back(A[i]);
00229                 }
00230                 else {
00231                     kietiakai.push_back(A[i]);
00232                 }
00233             }
00234         }
00235         else if(divide_strategy == 2) {
00236             vargsiukai.reserve(6000000);
00237             for(auto it = A.end(); it != A.begin(); ) {
00238                 it--;
00239                 if(it->rez() < 5) {
00240                     vargsiukai.push_back(*it);
00241                     it = A.erase(it);
00242                 }
00243             }
00244             std::reverse(vargsiukai.begin(), vargsiukai.end());
00245         }
00246         else if(divide_strategy == 3) {
00247             auto split_it = std::stable_partition(A.begin(), A.end(),
00248                 [](const Studentas& s) {
00249                     return s.rez() >= 5;
00250                 }
00251             );
00252             vargsiukai.reserve(std::distance(split_it, A.end()));
00253             for(auto it = split_it; it != A.end(); it++) {
00254                 vargsiukai.push_back(move(*it));
00255             }
00256             A.erase(split_it, A.end());
00257         }
00258         auto end2 = high_resolution_clock::now();
00259         duration<double> diff2 = end2 - start2;
00260         ofstream v_fout("vargsiukai.txt");
00261         ofstream k_fout("kietiakai.txt");
00262         auto start3 = high_resolution_clock::now();
00263         print(v_fout, grade_choice, vargsiukai, vargsiukai.size());
00264         auto end3 = high_resolution_clock::now();
00265         duration<double> diff3 = end3 - start3;
00266         auto start4 = high_resolution_clock::now();
00267         if(divide_strategy == 1) {
00268             print(k_fout, grade_choice, kietiakai, kietiakai.size());
00269         }
00270         else {
00271             print(k_fout, grade_choice, A, A.size());
00272         }
00273         auto end4 = high_resolution_clock::now();
00274         duration<double> diff4 = end4 - start4;

```

```

00275         v_fout.close();
00276         k_fout.close();
00277         cout << filename << " Failo skaitymo laikas: " << test_time << endl;
00278         cout << filename << " Rusiavimo laikas: " << diff1.count() << endl;
00279         cout << filename << " Vargsiuku ir kietiaku atskyrimo laikas: " << diff2.count() << endl;
00280         cout << filename << " Vargsiuku isvedimo i faila laikas: " << diff3.count() << endl;
00281         cout << filename << " Kietiaku isvedimo i faila laikas: " << diff4.count() << endl;
00282         cout << filename << " Testu laikas: " << test_time + diff1.count() + diff2.count() +
diff3.count() + diff4.count() << endl;
00283     }
00284     else {
00285         rusiavimas(A, sort_choice);
00286         int output_choice;
00287         output_choice = getInput<int,1,2>(
00288             "Kaip norite isvesti rezultatus? (1 - i ekrana, 2 - i faila) ",
00289             "Iveskite 1 arba 2."
00290         );
00291         auto start = high_resolution_clock::now();
00292         if(output_choice == 1) {
00293             print(cout, grade_choice, A, stud_skaicius);
00294         }
00295         else {
00296             ofstream fout("rezultatai.txt");
00297             print(fout, grade_choice, A, stud_skaicius);
00298             cout << "Rezultatai faile - rezultatai.txt" << endl;
00299             fout.close();
00300         }
00301         auto end = high_resolution_clock::now();
00302         duration<double> diff = end - start;
00303         cout << "Duomeni isvedimas uztruko: " << diff.count() << " sekundziu." << endl;
00304     }
00305 }

```

5.7 Person.h File Reference

```

#include <iostream>
#include <string>

```

Classes

- class **Zmogus**

5.8 Person.h

Go to the documentation of this file.

```

00001 #ifndef ZMOGUS_H
00002 #define ZMOGUS_H
00003
00004 #include <iostream>
00005 #include <string>
00006
00007 using std::string;
00008 using std::istream;
00009 using std::ostream;
00010
00011 class Zmogus {
00012 protected:
00013     string vardas_;
00014     string pavarde_;
00015
00016 public:
00017     Zmogus() : vardas_(), pavarde_() {};
00018     Zmogus(string vardas, string pavarde) : vardas_(vardas), pavarde_(pavarde) {}
00019
00020     virtual string vardas() const { return vardas_; }
00021     virtual string pavarde() const { return pavarde_; }
00022
00023     virtual double galBalas() const = 0;
00024     virtual istream& readStudent(istream& is) = 0;

```

```

00025     virtual void print(ostream& os) const = 0;
00026
00027     virtual ~Zmogus() {};
00028
00029     Zmogus(const Zmogus& other)
00030         : vardas_(other.vardas_), pavarde_(other.pavarde_) {};
00031
00032     Zmogus(Zmogus&& other) noexcept
00033         : vardas_(move(other.vardas_)), pavarde_(move(other.pavarde_)) {};
00034
00035     Zmogus& operator=(const Zmogus& other) {
00036         if(this == &other) return *this;
00037         vardas_ = other.vardas_;
00038         pavarde_ = other.pavarde_;
00039         return *this;
00040     }
00041     Zmogus& operator=(Zmogus&& other) noexcept {
00042         if(this == &other) return *this;
00043         vardas_ = move(other.vardas_);
00044         pavarde_ = move(other.pavarde_);
00045         return *this;
00046     }
00047
00048     friend istream& operator>>(istream& is, Zmogus& z) {
00049         return z.readStudent(is);
00050     }
00051     friend ostream& operator<<(ostream& os, const Zmogus& z) {
00052         z.print(os);
00053         return os;
00054     }
00055 };
00056
00057 #endif

```

5.9 README.md File Reference

5.10 Studentas.cpp File Reference

```

#include "Studentas.h"
#include <sstream>

```

Functions

- double **mediana** (std::vector< int > &paz)
- double **vidurkis** (std::vector< int > &paz)
- bool **compare** (const **Studentas** &a, const **Studentas** &b)
- bool **comparePagalPavarde** (const **Studentas** &a, const **Studentas** &b)
- bool **comparePagalRez** (const **Studentas** &a, const **Studentas** &b)
- void **rusiavimas** (vector< **Studentas** > &A, int sort_choice)
- void **print** (ostream &os, int pasirinkimas, const vector< **Studentas** > &A, int stud_skaicius)

5.10.1 Function Documentation

5.10.1.1 compare()

```

bool compare (
    const Studentas & a,
    const Studentas & b)

```

Definition at line **139** of file **Studentas.cpp**.

5.10.1.2 comparePagalPavarde()

```
bool comparePagalPavarde (
    const Studentas & a,
    const Studentas & b)
```

Definition at line 143 of file **Studentas.cpp**.

5.10.1.3 comparePagalRez()

```
bool comparePagalRez (
    const Studentas & a,
    const Studentas & b)
```

Definition at line 147 of file **Studentas.cpp**.

5.10.1.4 mediana()

```
double mediana (
    std::vector< int > & paz)
```

Definition at line 20 of file **Studentas.cpp**.

5.10.1.5 print()

```
void print (
    ostream & os,
    int pasirinkimas,
    const vector< Studentas > & A,
    int stud_skaicius)
```

Definition at line 163 of file **Studentas.cpp**.

5.10.1.6 rusiavimas()

```
void rusiavimas (
    vector< Studentas > & A,
    int sort_choice)
```

Definition at line 151 of file **Studentas.cpp**.

5.10.1.7 vidurkis()

```
double vidurkis (
    std::vector< int > & paz)
```

Definition at line 35 of file **Studentas.cpp**.

5.11 Studentas.cpp

Go to the documentation of this file.

```

00001 #include "Studentas.h"
00002 #include <sstream>
00003
00004 Studentas::Studentas() : Zmogus("Test", "Test") {
00005     paz_ = { 0 };
00006     egzam_ = 0;
00007     skaiciuotiRez();
00008 }
00009
00010 Studentas::Studentas(string vardas, string pavarde, vector<int> paz, double egzam) : Zmogus(vardas,
00011     pavarde) {
00012     paz_ = paz;
00013     egzam_ = egzam;
00014     skaiciuotiRez();
00015 }
00016 Studentas::Studentas(istream& is) : Zmogus() {
00017     readStudent(is);
00018 }
00019
00020 double mediana(std::vector<int>& paz) {
00021     double med;
00022     if (paz.empty()) {
00023         return 0;
00024     }
00025     std::sort(paz.begin(), paz.end());
00026     if (paz.size() % 2 == 0) {
00027         med = (paz[paz.size() / 2 - 1] + paz[paz.size() / 2]) / 2.0;
00028     }
00029     else {
00030         med = paz[paz.size() / 2];
00031     }
00032     return med;
00033 }
00034
00035 double vidurkis(std::vector<int>& paz) {
00036     if (paz.empty()) {
00037         return 0;
00038     }
00039     double sum = 0;
00040     for (int j = 0; j < paz.size(); j++) {
00041         sum += paz[j];
00042     }
00043     return sum / paz.size();
00044 }
00045
00046 double Studentas::galBalas() const {
00047     if (paz_.empty()) {
00048         return egzam_ * 0.6;
00049     }
00050     else {
00051         vector<int> kopija;
00052         kopija.reserve(paz_.size());
00053         for (int pazymys : paz_) {
00054             kopija.push_back(pazymys);
00055         }
00056         return mediana(kopija) * 0.4 + egzam_ * 0.6;
00057     }
00058 }
00059
00060 double Studentas::galBalas(double (*strategija)(vector<int>&)) const {
00061     if (paz_.empty()) {
00062         return egzam_ * 0.6;
00063     }
00064     else {
00065         vector<int> kopija;
00066         kopija.reserve(paz_.size());
00067         for (int pazymys : paz_) {
00068             kopija.push_back(pazymys);
00069         }
00070         return strategija(kopija) * 0.4 + egzam_ * 0.6;
00071     }
00072 }
00073
00074 void Studentas::skaiciuotiRez(double (*strategija)(vector<int>&)) {
00075     rez_ = galBalas(strategija);
00076 }
00077
00078 istream& Studentas::readStudent(istream& is) {
00079     paz_.clear();
00080     egzam_ = 0.0;
00081     rez_ = 0.0;

```



```

00082     string line;
00083     if (!getline(is » ws, line))
00084         return is;
00085     stringstream ss(line);
00086     if (!(ss » vardas_ » pavarde_))
00087         return is;
00088     int val;
00089     while (ss » val) {
00090         paz_.push_back(val);
00091     }
00092     if (!paz_.empty()) {
00093         egzam_ = paz_.back();
00094         paz_.pop_back();
00095     }
00096     return is;
00097 }
00098
00099 Studentas::~Studentas() {} // destructor
00100
00101 Studentas::Studentas(const Studentas& other) : Zmogus(other) { // copy constructor
00102     paz_ = other.paz_;
00103     egzam_ = other.egzam_;
00104     rez_ = other.rez_;
00105 }
00106
00107 Studentas::Studentas(Studentas&& other) noexcept : Zmogus(move(other)) { // move constructor
00108     paz_ = move(other.paz_);
00109     egzam_ = other.egzam_;
00110     rez_ = other.rez_;
00111     other.egzam_ = 0.0;
00112     other.rez_ = 0.0;
00113 }
00114 Studentas& Studentas::operator=(const Studentas& other) { // copy assignment
00115     if (this == &other) return *this;
00116     Zmogus::operator=(other);
00117     paz_ = other.paz_;
00118     egzam_ = other.egzam_;
00119     rez_ = other.rez_;
00120     return *this;
00121 }
00122
00123 Studentas& Studentas::operator=(Studentas&& other) noexcept { // move assignment
00124     if (this == &other) return *this;
00125     Zmogus::operator=(move(other));
00126     paz_ = move(other.paz_);
00127     egzam_ = other.egzam_;
00128     rez_ = other.rez_;
00129     other.egzam_ = 0.0;
00130     other.rez_ = 0.0;
00131     return *this;
00132 }
00133
00134 void Studentas::print(ostream& os) const {
00135     os « left « setw(15) « vardas() « left « setw(20) « pavarde();
00136     os « setw(10) « fixed « setprecision(2) « rez();
00137 }
00138
00139 bool compare(const Studentas& a, const Studentas& b) {
00140     return a.vardas() < b.vardas();
00141 }
00142
00143 bool comparePagalPavarde(const Studentas& a, const Studentas& b) {
00144     return a.pavarde() < b.pavarde();
00145 }
00146
00147 bool comparePagalRez(const Studentas& a, const Studentas& b) {
00148     return a.rez() > b.rez();
00149 }
00150
00151 void rusiavimas(vector<Studentas>& A, int sort_choice) {
00152     if (sort_choice == 1) {
00153         sort(A.begin(), A.end(), compare);
00154     }
00155     else if (sort_choice == 2) {
00156         sort(A.begin(), A.end(), comparePagalPavarde);
00157     }
00158     else if (sort_choice == 3) {
00159         sort(A.begin(), A.end(), comparePagalRez);
00160     }
00161 }
00162
00163 void print(ostream& os, int pasirinkimas, const vector<Studentas>& A, int stud_skaicius) {
00164     os « left « setw(15) « "Vardas" « setw(20) « "Pavarde";
00165     (pasirinkimas == 1)? os « setw(10) « "Galutinis (Vid.)" « endl : os « setw(10) « "Galutinis
(Med.)" « endl;
00166
00167     for (int i = 0; i < stud_skaicius; i++) {

```

```
00168         os « A[i] « endl;
00169     }
00170 }
00171
00172 string Studentas::test_eilute() {
00173     string eil;
00174     eil += vardas_ + " " + pavarde_;
00175     return eil;
00176 }
00177
```

5.12 Studentas.h File Reference

```
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
#include <stdexcept>
#include <numeric>
#include <iomanip>
#include "Person.h"
```

Classes

- class **Studentas**

Functions

- double **mediana** (vector< int > &paz)
- double **vidurkis** (vector< int > &paz)
- bool **compare** (const **Studentas** &a, const **Studentas** &b)
- bool **comparePagalPavarde** (const **Studentas** &a, const **Studentas** &b)
- bool **comparePagalRez** (const **Studentas** &a, const **Studentas** &b)
- void **rusiavimas** (vector< **Studentas** > &A, int sort_choice)
- void **print** (ostream &os, int pasirinkimas, const vector< **Studentas** > &A, int stud_skaicius)
- void **outputas** (vector< **Studentas** > &A, int stud_skaicius, double test_time, string filename)

5.12.1 Function Documentation

5.12.1.1 compare()

```
bool compare (
    const Studentas & a,
    const Studentas & b)
```

Definition at line 139 of file **Studentas.cpp**.

5.12.1.2 comparePagalPavarde()

```
bool comparePagalPavarde (  
    const Studentas & a,  
    const Studentas & b)
```

Definition at line 143 of file **Studentas.cpp**.

5.12.1.3 comparePagalRez()

```
bool comparePagalRez (  
    const Studentas & a,  
    const Studentas & b)
```

Definition at line 147 of file **Studentas.cpp**.

5.12.1.4 mediana()

```
double mediana (  
    vector< int > & paz)
```

5.12.1.5 outputas()

```
void outputas (  
    vector< Studentas > & A,  
    int stud_skaicius,  
    double test_time,  
    string filename)
```

Definition at line 184 of file **main.cpp**.

5.12.1.6 print()

```
void print (  
    ostream & os,  
    int pasirinkimas,  
    const vector< Studentas > & A,  
    int stud_skaicius)
```

Definition at line 163 of file **Studentas.cpp**.

5.12.1.7 rusiavimas()

```
void rusiavimas (  
    vector< Studentas > & A,  
    int sort_choice)
```

Definition at line 151 of file **Studentas.cpp**.

5.12.1.8 vidurkis()

```
double vidurkis (
    vector< int > & paz)
```

5.13 Studentas.h

Go to the documentation of this file.

```
00001 #ifndef STUDENTAS_H
00002 #define STUDENTAS_H
00003
00004 #include <iostream>
00005 #include <string>
00006 #include <vector>
00007 #include <algorithm>
00008 #include <stdexcept>
00009 #include <numeric>
00010 #include <iomanip>
00011 #include "Person.h"
00012
00013 using std::string;
00014 using std::vector;
00015 using std::istream;
00016 using std::ostream;
00017 using std::sort;
00018 using std::move;
00019 using std::endl;
00020 using std::left;
00021 using std::setw;
00022 using std::fixed;
00023 using std::setprecision;
00024 using std::stringstream;
00025 using std::ws;
00026
00027 double mediana(vector<int>& paz);
00028 double vidurkis(vector<int>& paz);
00029
00030
00031 class Studentas : public Zmogus {
00032 private:
00033     vector<int> paz_;
00034     double egzam_;
00035     double rez_;
00036
00037 public:
00038     Studentas(); // default constructor
00039     Studentas(string vardas, string pavarde, vector<int> paz, double egzam); // parameterized
00040     constructor
00041     Studentas(istream& is);
00042
00043     double egzam() const { return egzam_; }
00044     double rez() const { return rez_; }
00045     const vector<int>& paz() const { return paz_; }
00046
00047     double galBalas() const override;
00048     void print(ostream& os) const override;
00049
00050     double galBalas(double (*) (vector<int>&) = mediana) const;
00051     void skaiciuotiRez(double (*) (vector<int>&) = mediana);
00052
00053     ~Studentas() override; // destructor
00054     Studentas(const Studentas& other); // copy constructor
00055     Studentas(Studentas&& other) noexcept; // move constructor
00056     Studentas& operator=(const Studentas& other); // copy assignment
00057     Studentas& operator=(Studentas&& other) noexcept; // move assignment
00058     istream& readStudent(istream& is) override;
00059
00060     string test_eilute();
00061 };
00062
00063 bool compare(const Studentas& a, const Studentas& b);
00064 bool comparePagalPavarde(const Studentas& a, const Studentas& b);
00065 bool comparePagalRez(const Studentas& a, const Studentas& b);
00066
00067 void rusiavimas(vector<Studentas>& A, int sort_choice);
00068 void print(ostream& os, int pasirinkimas, const vector<Studentas>& A, int stud_skaicius);
00069 void outputas(vector<Studentas> &A, int stud_skaicius, double test_time, string filename);
00070 #endif
```

5.14 Test.cpp File Reference

```
#include <gtest/gtest.h>
#include <sstream>
#include "Studentas.h"
```

Functions

- **TEST** (StudentasTest, DefaultConstructor)
- **TEST** (StudentasTest, ParameterizedConstructor)
- **TEST** (StudentasTest, CopyConstructor)
- **TEST** (StudentasTest, CopyAssignment)
- **TEST** (StudentasTest, MoveConstructor)
- **TEST** (StudentasTest, MoveAssignment)
- **TEST** (StudentasTest, Getters)
- **TEST** (StudentasTest, Destructor)
- **TEST** (StudentasTest, InputOperator)
- **TEST** (StudentasTest, OutputOperator)
- int **main** (int argc, char **argv)

5.14.1 Function Documentation

5.14.1.1 main()

```
int main (
    int argc,
    char ** argv)
```

Definition at line 74 of file **Test.cpp**.

5.14.1.2 TEST() [1/10]

```
TEST (
    StudentasTest ,
    CopyAssignment )
```

Definition at line 21 of file **Test.cpp**.

5.14.1.3 TEST() [2/10]

```
TEST (
    StudentasTest ,
    CopyConstructor )
```

Definition at line 15 of file **Test.cpp**.

5.14.1.4 TEST() [3/10]

```
TEST (
    StudentasTest ,
    DefaultConstructor )
```

Definition at line 5 of file **Test.cpp**.

5.14.1.5 TEST() [4/10]

```
TEST (
    StudentasTest ,
    Destructor )
```

Definition at line 50 of file **Test.cpp**.

5.14.1.6 TEST() [5/10]

```
TEST (
    StudentasTest ,
    Getters )
```

Definition at line 43 of file **Test.cpp**.

5.14.1.7 TEST() [6/10]

```
TEST (
    StudentasTest ,
    InputOperator )
```

Definition at line 57 of file **Test.cpp**.

5.14.1.8 TEST() [7/10]

```
TEST (
    StudentasTest ,
    MoveAssignment )
```

Definition at line 35 of file **Test.cpp**.

5.14.1.9 TEST() [8/10]

```
TEST (
    StudentasTest ,
    MoveConstructor )
```

Definition at line 28 of file **Test.cpp**.

5.14.1.10 TEST() [9/10]

```
TEST (
    StudentasTest ,
    OutputOperator )
```

Definition at line 65 of file **Test.cpp**.

5.14.1.11 TEST() [10/10]

```
TEST (
    StudentasTest ,
    ParameterizedConstructor )
```

Definition at line 10 of file **Test.cpp**.

5.15 Test.cpp

Go to the documentation of this file.

```
00001 #include <gtest/gtest.h>
00002 #include <sstream>
00003 #include "Studentas.h"
00004
00005 TEST(StudentasTest, DefaultConstructor)
00006 {
00007     Studentas S;
00008     EXPECT_EQ(S.test_eilute(), Studentas("Test", "Test", { 0 }, 0).test_eilute());
00009 }
00010 TEST(StudentasTest, ParameterizedConstructor)
00011 {
00012     Studentas S("Vardenis", "Pavardenis", { 8,9,7 }, 8);
00013     EXPECT_EQ(S.test_eilute(), Studentas("Vardenis", "Pavardenis", { 8,9,7 }, 8).test_eilute());
00014 }
00015 TEST(StudentasTest, CopyConstructor)
00016 {
00017     Studentas S1("Vardenis", "Pavardenis", { 8,9,7 }, 8);
00018     Studentas S2(S1);
00019     EXPECT_EQ(S1.test_eilute(), S2.test_eilute());
00020 }
00021 TEST(StudentasTest, CopyAssignment)
00022 {
00023     Studentas S1("Vardenis", "Pavardenis", { 8,9,7 }, 8);
00024     Studentas S2;
00025     S2 = S1;
00026     EXPECT_EQ(S1.test_eilute(), S2.test_eilute());
00027 }
00028 TEST(StudentasTest, MoveConstructor)
00029 {
00030     Studentas S1("Vardenis", "Pavardenis", { 8,9,7 }, 8);
00031     Studentas S2(std::move(S1));
00032     EXPECT_EQ(S2.test_eilute(), Studentas("Vardenis", "Pavardenis", { 8,9,7 }, 8).test_eilute());
00033     EXPECT_NE(S1.test_eilute(), S2.test_eilute());
00034 }
00035 TEST(StudentasTest, MoveAssignment)
00036 {
00037     Studentas S1("Vardenis", "Pavardenis", { 8,9,7 }, 8);
00038     Studentas S2;
00039     S2 = std::move(S1);
00040     EXPECT_EQ(S2.test_eilute(), Studentas("Vardenis", "Pavardenis", { 8,9,7 }, 8).test_eilute());
00041     EXPECT_NE(S1.test_eilute(), S2.test_eilute());
00042 }
00043 TEST(StudentasTest, Getters)
00044 {
00045     Studentas S("Vardenis", "Pavardenis", { 8,9,7 }, 8);
00046     EXPECT_STREQ(S.vardas().c_str(), "Vardenis");
00047     EXPECT_STREQ(S.pavarde().c_str(), "Pavardenis");
00048     EXPECT_DOUBLE_EQ(S.egzam(), 8.0);
00049 }
00050 TEST(StudentasTest, Destructor)
00051 {
```

```

00052     {
00053         Studentas S("Vardenis", "Pavardenis", { 8,9,7 }, 8);
00054     }
00055     SUCCEED();
00056 }
00057 TEST(StudentasTest, InputOperator)
00058 {
00059     std::istringstream iss("Vardenis Pavardenis 8 9 7 8");
00060     Studentas S;
00061     iss >> S;
00062     EXPECT_EQ(S.test_eilute(), Studentas("Vardenis", "Pavardenis", { 8,9,7 }, 8).test_eilute());
00063     EXPECT_DOUBLE_EQ(S.egzam(), 8.0);
00064 }
00065 TEST(StudentasTest, OutputOperator)
00066 {
00067     Studentas S("Vardenis", "Pavardenis", { 8,9,7 }, 8);
00068     std::ostringstream oss;
00069     oss << S;
00070     std::string expected = "Vardenis      Pavardenis      8.00      ";
00071     EXPECT_EQ(oss.str(), expected);
00072 }
00073
00074 int main(int argc, char **argv) {
00075     ::testing::InitGoogleTest(&argc, argv);
00076     return RUN_ALL_TESTS();
00077 }

```

5.16 Zmones.cpp File Reference

```
#include "Zmones.h"
```

Functions

- `zmogus gen()`

5.16.1 Function Documentation

5.16.1.1 `gen()`

zmogus `gen()`

Definition at line 3 of file `Zmones.cpp`.

5.17 Zmones.cpp

Go to the documentation of this file.

```

00001 #include "Zmones.h"
00002
00003 zmogus gen() {
00004     zmogus a;
00005
00006     switch(rand() % 10) {
00007         case 0: a.vardas = "Jonas"; break;
00008         case 1: a.vardas = "Petras"; break;
00009         case 2: a.vardas = "Dainius"; break;
00010         case 3: a.vardas = "Rokas"; break;
00011         case 4: a.vardas = "Marius"; break;
00012         case 5: a.vardas = "Irma"; break;
00013         case 6: a.vardas = "Lina"; break;
00014         case 7: a.vardas = "Giedre"; break;
00015         case 8: a.vardas = "Toma"; break;

```



```

00016         case 9: a.vardas = "Laura"; break;
00017     }
00018     switch(*a.vardas.rbegin()) {
00019         case 's':
00020             switch(rand() % 5) {
00021                 case 0: a.pavarde = "Pavardenis1"; break;
00022                 case 1: a.pavarde = "Pavardenis2"; break;
00023                 case 2: a.pavarde = "Pavardenis3"; break;
00024                 case 3: a.pavarde = "Pavardenis4"; break;
00025                 case 4: a.pavarde = "Pavardenis5"; break;
00026             }
00027             break;
00028         default:
00029             switch(rand() % 5) {
00030                 case 0: a.pavarde = "Pavardaitel"; break;
00031                 case 1: a.pavarde = "Pavardaitel2"; break;
00032                 case 2: a.pavarde = "Pavardaitel3"; break;
00033                 case 3: a.pavarde = "Pavardaitel4"; break;
00034                 case 4: a.pavarde = "Pavardaitel5"; break;
00035             }
00036             break;
00037     }
00038     return a;
00039 }

```

5.18 Zmones.h File Reference

```

#include <iostream>
#include <string>
#include <cstdlib>

```

Classes

- struct **zmogus**

Functions

- **zmogus** gen ()

5.18.1 Function Documentation

5.18.1.1 gen()

zmogus gen ()

Definition at line 3 of file **Zmones.cpp**.

5.19 Zmones.h

Go to the documentation of this file.

```

00001 #ifndef ZMONES_H
00002 #define ZMONES_H
00003
00004 #include <iostream>
00005 #include <string>
00006 #include <cstdlib>
00007
00008 using std :: string;
00009
00010 struct zmogus {
00011     string vardas;
00012     string pavarde;
00013 };
00014
00015 zmogus gen();
00016
00017 #endif

```


Index

- ~Studentas
 - Studentas, 8
- ~Zmogus
 - Zmogus, 13
- clearInput
 - Funkcijos.cpp, 17
 - Funkcijos.h, 23
- compare
 - Funkcijos.h, 23
 - Studentas.cpp, 32
 - Studentas.h, 36
- comparePagalPavarde
 - Studentas.cpp, 32
 - Studentas.h, 36
- comparePagalRez
 - Studentas.cpp, 33
 - Studentas.h, 37
- egzam
 - Studentas, 9
 - studentas, 11
- failu_generavimas
 - Funkcijos.cpp, 17
 - Funkcijos.h, 23
- Funkcijos.cpp, 17
 - clearInput, 17
 - failu_generavimas, 17
 - getWordInput, 18
 - mediana, 18
 - print, 18
 - raide_choice, 18
 - rusiavimas, 19
 - vidurkis, 19
- Funkcijos.h, 22
 - clearInput, 23
 - compare, 23
 - failu_generavimas, 23
 - getInput, 23
 - getWordInput, 23
 - mediana, 23
 - outputas, 24
 - print, 24
 - raide_choice, 25
 - rusiavimas, 25
 - vidurkis, 25
- galBalas
 - Studentas, 9
- Zmogus, 14
- gen
 - Zmones.cpp, 42
 - Zmones.h, 43
- getInput
 - Funkcijos.h, 23
- getWordInput
 - Funkcijos.cpp, 18
 - Funkcijos.h, 23
- main
 - main.cpp, 27
 - Test.cpp, 39
- main.cpp, 26
 - main, 27
 - outputas, 27
- mediana
 - Funkcijos.cpp, 18
 - Funkcijos.h, 23
 - Studentas.cpp, 33
 - Studentas.h, 37
- operator<<
 - Zmogus, 15
- operator>>
 - Zmogus, 15
- operator=
 - Studentas, 9
 - Zmogus, 14
- outputas
 - Funkcijos.h, 24
 - main.cpp, 27
 - Studentas.h, 37
- pavarde
 - studentas, 11
 - Zmogus, 14
 - zmogus, 16
- pavarde_
 - Zmogus, 15
- paz
 - Studentas, 10
 - studentas, 11
- Person.h, 31
- print
 - Funkcijos.cpp, 18
 - Funkcijos.h, 24
 - Studentas, 10
 - Studentas.cpp, 33
 - Studentas.h, 37

- Zmogus, 14
- raide_choice
 - Funkcijos.cpp, 18
 - Funkcijos.h, 25
- README.md, 32
- readStudent
 - Studentas, 10
 - Zmogus, 14
- rez
 - Studentas, 10
 - studentas, 11
- rusiavimas
 - Funkcijos.cpp, 19
 - Funkcijos.h, 25
 - Studentas.cpp, 33
 - Studentas.h, 37
- skaiciuotiRez
 - Studentas, 10
- Studentas, 7
 - ~Studentas, 8
 - egzam, 9
 - galBalas, 9
 - operator=, 9
 - paz, 10
 - print, 10
 - readStudent, 10
 - rez, 10
 - skaiciuotiRez, 10
 - Studentas, 8, 9
 - test_eilute, 10
- studentas, 11
 - egzam, 11
 - pavarde, 11
 - paz, 11
 - rez, 11
 - vardas, 12
- Studentas.cpp, 32
 - compare, 32
 - comparePagalPavarde, 32
 - comparePagalRez, 33
 - mediana, 33
 - print, 33
 - rusiavimas, 33
 - vidurkis, 33
- Studentas.h, 36
 - compare, 36
 - comparePagalPavarde, 36
 - comparePagalRez, 37
 - mediana, 37
 - outputas, 37
 - print, 37
 - rusiavimas, 37
 - vidurkis, 37
- TEST
 - Test.cpp, 39–41
- Test.cpp, 39
 - main, 39
 - TEST, 39–41
- test_eilute
 - Studentas, 10
- vardas
 - studentas, 12
 - Zmogus, 14
 - zmogus, 16
- vardas_
 - Zmogus, 15
- vidurkis
 - Funkcijos.cpp, 19
 - Funkcijos.h, 25
 - Studentas.cpp, 33
 - Studentas.h, 37
- Zmogus, 12
 - ~Zmogus, 13
 - galBalas, 14
 - operator<<, 15
 - operator>>, 15
 - operator=, 14
 - pavarde, 14
 - pavarde_, 15
 - print, 14
 - readStudent, 14
 - vardas, 14
 - vardas_, 15
 - Zmogus, 13
- zmogus, 15
 - pavarde, 16
 - vardas, 16
- Zmones.cpp, 42
 - gen, 42
- Zmones.h, 43
 - gen, 43